

Toyl Language Reference v0.1

Sai Hemanth Bheemreddy

May 2026

Chapter 1

Spec for V0.1

1.1 Statically & Strongly Typed and Compiled

Toyl v0.1 will be a Statically and Strongly Typed. All variables will be allocated on Stack or in Data Section. Types will not be implicitly converted, i.e., *int* + *float* will not result in the *int* being promoted to *float*.

1.2 No heap or memory management

To keep the implementation simple and manageable, v0.1 will not have any support for heap memory or any kind of runtime memory allocation. Eventually when this feature is added, Toyl will use Rust style Ownership+Swift style ARC to do automatic memory management.

1.3 Builtin Datatypes

v0.1 will only support *int*, *float*, *boolean*, *nil* and *constant-size array*

- *int*: Toyl v0.1 has 4 int subtypes: **i8**, Numbers can start with: A digit, +/- signs, **0x** or **0X** for hexadecimal, **0o** or **0O** for Octal, **0b** or **0B** for Binary.
- *float*: Toyl v0.1 has 3 subtypes: **fp16**, **fp32**, **fp64**. Floating point numbers can contain digits, period and +/- signs. If a floating-point number doesn't contain the decimal part, it can be suffixed with **f** or **F** to explicitly specify the literal is a floating-point number.
- *boolean*: Defined using **bool**. Can be either **true** or **false**. Other types are not implicitly converted to Boolean, for example, **0** is not converted to **false**
- *nil*: This is a temp. builtin type to allow variables to be null or nil. Later versions will not allow nil values for variables. All variables must have a defined values.
- *constant-size array*: Constant fixed size arrays of other "scaler" datatypes.

1.4 Lexical Rules

Token Type	Token Value
Keywords	
kw_fn	fn
kw_var	var
kw_for	for
kw_in	in
kw_while	while
kw_if	if
kw_else	else
kw_break	break
kw_continue	continue
kw_return	return
kw_nil	nil
kw_true	true
kw_false	false
Literals	
identifier	/[A-Za-z_][A-Za-z0-9_]*-/
int_literal	1.4.1
float_literal	1.4.2
singleline_comment	/* */
multiline_comment	/*[s\S]*?*/
Miscellaneous	
tk_eof	End of File
tk_unknown	Unknown Token

Token Type	Token Value
Punctuation	
colon	:
arrow	->
comma	,
semicolon	;
l_param	(
r_param	)
l_brace	{
r_brace	}
l_square	[
r_square	]
equal	=
equalequal	==
plus	+
minus	-
star	*
slash	/
percent	%
less	<
lesequal	<=
greater	>
greaterequal	>=
exclaim	!
exclaimequal	!=
ampamp	&&
pipepipe	
in_range	...
ex_range	..<

1.4.1 Integer Literal: /(0[xX][0-9a-f]+|0[bB][01]+|0[oO][0-7]+|[0-9]+)/

1.4.2 Float Literal: /([0-9]+[0-9]+([eE][+-]?[0-9]+)?[fF]?|[0-9]+[eE][+-]?[0-9]+[fF]?|[0-9]+[fF])/

1.5 Grammar

1.6 Semantic Rules

1.7 Data Types